

Free and Low-Cost Sources and a Practical Pipeline for Level 1 Word Packages

Executive summary

For a **Level 1 pilot of 20 to 80 words**, the most practical low-cost stack is a **hybrid** one: use your **owned Oxford 3000 CSV** as the authority for level placement and POS, use **FreeDictionaryAPI.com** or **Wiktionary dumps via Wikitextract** for open lexical content, use **Datamuse** and **WordNet** for distractors and semantic checks, and generate your own stable audio with **local TTS** or a cloud free tier. This gives you enough structure to build package skeletons and Supabase rows without paying for a full learner-dictionary license on day one. ¹

The key caveat is **license safety**. The cleanest free/open lexical sources in this space are often **CC BY-SA** or otherwise require attribution. FreeDictionaryAPI.com explicitly says its content comes from **Wiktionary under CC BY-SA 4.0** and requires visible attribution and a link back to the original Wiktionary page. CC-CEDICT is also **CC BY-SA 4.0**. That means if you ship verbatim or adapted text from those sources, your final distributed material can trigger **ShareAlike** obligations. For a prototype, that may be acceptable if you isolate those fields and show attribution. For a closed proprietary production app, the safer path is to use the free/open stack for **drafting, QA, and internal authoring support**, then replace or rewrite final learner-facing definitions/examples with original editorial copy or licensed proprietary data. ²

Your hardest free-data problem is not level mapping; it is **high-quality learner-friendly definitions and example sentences**. WordNet is permissive and useful, but it only covers open-class words and is not learner-graded. Wiktionary-derived data is broad and often rich, but its style is inconsistent for beginners. That makes an **AI or editorial rewrite step** essential if your goal is A1/A2-friendly explanations rather than “dictionary-complete” explanations. Merriam-Webster’s Learner’s Dictionary, Cambridge, and Oxford all offer better learner-style content, but public production pricing is limited or quote-based except for Oxford’s public API plan. ³

The short recommendation is this: **pilot with open sources, your own TTS, strict provenance tracking, and human QA; only license a paid dictionary API if the pilot proves product fit or if proprietary distribution is a hard requirement from day one.** ⁴

Source comparison

Ranked source list

The table below prioritizes sources by **practical usefulness for your pipeline, cost, and license clarity**.

Rank	Source	Free	Rate limit or access	Fields available	License and attribution	Pros
1	Oxford 3000 CSV already owned	Already in hand	Local file	CEFR, ordering, topic/sublevel, source POS	Your project asset	Best starting authority for level placement and game progression
2	FreeDictionaryAPI.com 5	Yes	1,000 requests/hour/IP , no API key 5	Pronunciations, forms, POS, senses, examples, synonyms, antonyms, optional translations, source URL/ license in response 5	Data from Wiktionary, CC BY-SA 4.0 ; visible attribution required 5	Best open structured response for automation; explicit rate limit; explicit source and license metadata
3	Princeton WordNet 6	Yes	Downloadable / local	Synsets, glosses, examples, relations, POS for open-class words 7	Princeton license permits commercial use with notice preservation 8	Very strong for semantic checks and distractor sanity; permissive license
4	Datamuse 10	Yes	Up to 100,000 requests/day ; API key required starting 2027-01-01 10	Related words, synonyms, antonyms, hypernyms, followers, pronunciation guesses, frequency tags, optional definitions metadata 10	Free service; asks for acknowledgment in app docs 10	Excellent for distractor generation and word-neighborhood checks; easy JSON

Rank	Source	Free	Rate limit or access	Fields available	License and attribution	Pros
5	Wiktionary dumps + Wiktextextract ¹¹	Yes	Batch download / local processing	Broad multilingual data; forms, glosses, examples, translations, pronunciation-related extraction depending on language/ template support ¹²	Wiktionary content is CC BY-SA; Wiktextextract code is MIT except test files under Wiktionary license notes ¹³	Best large-scale self-hosted route; avoids API fragility; reproducible monthly refreshes
6	CC-CEDICT ¹⁵	Yes	Download release file manually	Chinese forms, pinyin, English glosses in CEDICT format; useful for validation and reverse lookup support ¹⁵	CC BY-SA 4.0 ; MDBG notes attribution and ShareAlike; MDBG webpage also says automated/ scripted access is prohibited ¹⁵	Excellent for validating Chinese strings, Chinese variants, and pinyin
7	dictionaryapi.dev ¹⁶	Yes	Public endpoint; no published limit on landing page reviewed	<code>phonetic</code> , <code>phonetics[]</code> , audio URLs, origin, meanings, definitions, examples, synonyms, antonyms ¹⁷	The repo is GPL-3.0 and describes the project as a Google-dictionary scraper; the site does not clearly publish dictionary-content licensing terms ¹⁸	Very easy bootstrap; good response shape for quick prototyping

Rank	Source	Free	Rate limit or access	Fields available	License and attribution	Pros
8	OPUS OpenSubtitles-derived frequency signals ¹⁹	Yes	Corpus download / local processing	Large subtitle corpus; useful to compute conversational frequency or familiarity signals ²⁰	Corpus provenance is subtitle-based and copyright handling is more complex; OpenSubtitles runs DMCA removal processes ²¹	Helpful as a ranking signal for “common in spoken media”

What this means for your project

If you want the **best free/open prototype stack**, the most balanced choice is: **Oxford CSV + FreeDictionaryAPI + WordNet + Datamuse + your own TTS**. That stack covers level placement, POS-aware sense extraction, fallback definitions, distractor generation, and audio generation at near-zero marginal cost. ²²

If you want the **lowest legal friction for a proprietary production product**, the open stack is still useful for the pilot, but you should treat Wiktionary- and CC-CEDICT-derived text as either **attributed open-content fields** or **replaceable drafts**, not as permanent closed-content assets. In that scenario, the minimal paid upgrade worth considering first is **Oxford Dictionaries API**, because it has a public sandbox and a public entry-level plan; Cambridge and Merriam-Webster require approval or quote-driven business terms for commercial use. ²³

Recommended hybrid pipeline

Recommended architecture

For your first two-week pilot, I would build a **two-layer content system**:

Layer one is a **staging package** that stores raw external material with provenance.

Layer two is a **shipping package** that stores your normalized, learner-facing, app-ready fields.

That separation gives you two benefits. First, it preserves auditability. Second, it makes it easy later to replace a risky source without redoing your entire level design. This is especially important because your most practical free sources are mostly **Wiktionary-derived and CC BY-SA**. ²⁴

A practical pilot pipeline looks like this:

```
Oxford CSV
→ choose 20 pilot lemmas
→ fetch candidate senses from FreeDictionaryAPI or Wiktionary dump extract
```

- match exact lemma + POS
- score and select one teaching sense
- rewrite to learner-friendly EN definition
- draft original EN example sentence
- generate / validate ZH translation
- generate distractors from Oxford level + Datamuse + WordNet
- generate mnemonic
- synthesize audio
- export Supabase CSVs + source audit

This hybrid design matches the strengths of each source: Oxford for progression, Wiktionary-derived data for breadth, WordNet and Datamuse for semantic control, and your own TTS for stable licensed audio. 22

Script order

Script	Input	Output	Pu
<code>build_level_seed.py</code>	<code>american_oxford_3000_a1.csv</code>	<code>level_001_seed.csv</code>	Se the 80 top an ma wo
<code>fetch_candidates.py</code>	seed CSV	<code>level_001_candidates.jsonl</code>	Pu fro Fre or Wi op fro dic
<code>normalize_senses.py</code>	candidates JSONL	<code>level_001_packages.todo.json</code>	Ma PC att ph ex tra
<code>rewrite_and_draft.py</code>	package TODO JSON	<code>level_001_packages.draft.json</code>	Cn fri de ori se Ch mr

Script	Input	Output	
<code>generate_distractors.py</code>	draft packages + seed list	<code>level_001_distractors.json</code>	Pu Bu dis lev Da Wo
<code>validate_packages.py</code>	draft packages	<code>level_001_packages.validated.json</code>	Ap an fla
<code>export_supabase.py</code>	validated packages	<code>words.csv</code> , <code>word_senses.csv</code> , <code>questions.csv</code> , <code>audio_manifest.csv</code>	Pr rea
<code>synthesize_audio.py</code>	audio manifest	audio files + <code>audio_index.csv</code>	Ge loc file

Validation rules that matter most

Your normalization rules should be strict, because free/open dictionary data is broad but noisy. The most important rule is **exact lemma plus POS matching** against your Oxford seed row. If Oxford says `book | n.`, do not allow a `book | v.` sense to win unless you have an explicit override. FreeDictionaryAPI exposes part of speech at the entry level, and WordNet organizes data by POS and synsets, which makes this rule implementable. ²⁵

For sense selection, prefer the candidate that satisfies most of these conditions: short definition, one clean example, frequent/neutral register, no archaic or highly domain-specific tags, and strong fit to a concrete Level 1 lesson. In practice, this means many perfectly valid dictionary senses should be rejected because they are too abstract, too rare, or too multi-step for a beginner package. Free/open dictionaries optimize for lexical coverage, not teaching sequence. ²⁵

For Chinese, do not rely on a single automatic source. The best low-cost pattern is: use **Wiktionary-derived sense translations** when available, then validate or normalize the selected Chinese wording with **CC-CEDICT** so you can standardize simplified/traditional form and pinyin if needed. Since CC-CEDICT is fundamentally Chinese→English, not a learner-ready English→Chinese sense bank, it works better as a validation layer than as your primary generator. ²⁶

For distractors, use **same POS, same or adjacent level band, and similar concreteness/topic**, but ban exact synonyms, direct antonyms, inflectional variants, and near-duplicates. Datamuse is ideal for proposing related-but-not-identical candidates, while WordNet helps you blacklist tricky semantic collisions.

²⁷

Automation examples and schemas

Example API mapping for dictionaryapi.dev

The official landing page shows a simple JSON structure with `phonetic`, `phonetics`, `meanings`, `definitions`, `example`, `synonyms`, and `antonyms`. That makes it easy to map quickly into your staging table even though I would still treat it as a **prototype or fallback source**, not your sole production authority. ¹⁶

```
GET https://api.dictionaryapi.dev/api/v2/entries/en/family
```

Suggested field mapping:

<code>response[0].word</code>	-> <code>headword</code>
<code>response[0].phonetic</code>	-> <code>phonetic_primary</code>
<code>response[0].phonetics[*].text</code>	-> <code>phonetic_variants</code>
<code>response[0].phonetics[*].audio</code>	-> <code>source_audio_urls</code>
<code>response[0].meanings[*].partOfSpeech</code>	-> <code>pos_candidates</code>
<code>response[0].meanings[*].definitions[*]</code>	-> <code>sense_candidates</code>
<code>definition</code>	-> <code>definition_raw</code>
<code>example</code>	-> <code>example_raw</code>
<code>synonyms</code>	-> <code>synonym_candidates</code>
<code>antonyms</code>	-> <code>antonym_candidates</code>
<code>origin</code>	-> <code>etymology_raw</code>

A good operational rule is to ingest all returned senses into staging, then run your own sense selector rather than trusting the first returned sense. ¹⁷

Example API mapping for Wiktionary

Wiktionary itself explicitly says there is **no standard data-only API** for its dictionary content; automated access is usually via **database dumps** or the MediaWiki **Action API**, which returns wikitext or HTML that you must parse yourself. That is why **Wiktextextract** is so useful for scale. ¹¹

For live fetches, the official MediaWiki approach is typically one of these:

```
GET https://en.wiktionary.org/w/api.php?
action=query&prop=revisions&titles=family&rvslots=*&rvprop=content&formatversion=2&format=json
```

```
GET https://en.wiktionary.org/w/api.php?action=parse&page=family&prop=wikitext|
text|tocdata&format=json
```

The MediaWiki docs show that `action=query` can return page wikitext via revisions, and `action=parse` can return parsed HTML or wikitext for a page. ²⁸

Suggested mapping for a direct live pull:

```
query.pages[0].revisions[0].slots.main.content    -> raw_wikitext
parse.wikitext["*"]                              -> raw_wikitext_alt
parse.text["*"]                                   -> raw_html_alt
```

Suggested mapping for a dump-based pipeline with Wiktextextract:

```
wiktextextract.word                -> headword
wiktextextract.pos                 -> pos
wiktextextract.senses[*].glosses   -> definition_raw
wiktextextract.senses[*].examples -> example_raw
wiktextextract.forms               -> word_forms
wiktextextract.translations        -> translation_candidates
wiktextextract.sounds or pronunciation fields -> phonetic_candidates
```

For your pilot, I would start with **FreeDictionaryAPI** because it already gives you structured JSON from Wiktionary-derived data. For scale, I would graduate to **monthly dump + Wiktextextract** so you control refresh timing and schema stability. ²⁹

Sample pilot workflow in Python

This is pseudocode, but it is close to an implementation pattern that will work for a 20-word pilot.

```
import csv
import json
from datetime import datetime
from typing import Any, Dict, List

def fetch_free_dictionary(headword: str) -> Dict[str, Any]:
    # GET FreeDictionaryAPI.com /v1/entries/en/<word>?translations=true
    ...

def fetch_datamuse_related(headword: str, max_items: int = 20) ->
List[Dict[str, Any]]:
    # GET Datamuse /words?ml=<word>&md=prf&max=<n>
    ...

def fetch_wordnet(headword: str, pos: str) -> List[Dict[str, Any]]:
    # local WordNet lookup via NLTK or downloaded DB
    ...
```

```

def coarse_pos(pos_raw: str) -> str:
    mapping = {
        "noun": "n", "verb": "v", "adjective": "adj", "adverb": "adv",
        "exclamation": "other", "preposition": "other", "pronoun": "other"
    }
    return mapping.get(pos_raw.lower(), "other")

def choose_best_sense(oxford_pos: str, fd_entry: Dict[str, Any]) -> Dict[str, Any]:
    candidates = []
    for entry in fd_entry.get("entries", []):
        if coarse_pos(entry.get("partOfSpeech", "")) != oxford_pos:
            continue
        for sense in entry.get("senses", []):
            score = 0
            definition = sense.get("definition", "").strip()
            examples = sense.get("examples", [])
            tags = sense.get("tags", [])
            if definition:
                score += 5
            if examples:
                score += 3
            if len(definition) < 120:
                score += 2
            if not any(t in {"archaic", "slang", "offensive"} for t in tags):
                score += 2
            candidates.append((score, entry, sense))
    if not candidates:
        return {}
    candidates.sort(key=lambda x: x[0], reverse=True)
    _, entry, sense = candidates[0]
    return {
        "pos": entry["partOfSpeech"],
        "definition_raw": sense.get("definition"),
        "example_raw": (sense.get("examples") or [""])[0],
        "translations_raw": sense.get("translations", []),
        "pronunciations_raw": entry.get("pronunciations", []),
        "forms_raw": entry.get("forms", []),
        "source_url": fd_entry.get("source", {}).get("url"),
        "license_name": fd_entry.get("source", {}).get("license",
    }).get("name"),
        "license_url": fd_entry.get("source", {}).get("license", {}).get("url"),
    }

def rewrite_for_learners(headword: str, definition_raw: str, example_raw: str) -
> Dict[str, str]:
    # LLM or editorial rewrite step:

```

```

# - definition_en: short, A1/A2-friendly, one sense only
# - sentence_en: short original example sentence
# - mnemonic: short, memorable, non-misleading
...

def build_source_audit(source_name: str, used_for_fields: List[str], raw_ref:
Dict[str, Any]) -> Dict[str, Any]:
    return {
        "source_name": source_name,
        "retrieved_at_utc": datetime.utcnow().isoformat() + "Z",
        "used_for_fields": used_for_fields,
        "raw_ref": raw_ref,
        "review_status": "pending"
    }

def process_seed(seed_csv: str) -> List[Dict[str, Any]]:
    packages = []
    with open(seed_csv, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            headword = row["headword"].strip()
            oxford_pos = row["pos"].strip().lower()

            fd = fetch_free_dictionary(headword)
            chosen = choose_best_sense(oxford_pos, fd)

            wn = fetch_wordnet(headword, oxford_pos)
            dm = fetch_datamuse_related(headword)

            draft = rewrite_for_learners(
                headword=headword,
                definition_raw=chosen.get("definition_raw", ""),
                example_raw=chosen.get("example_raw", "")
            )

            package = {
                "headword": headword,
                "cefr_level": row["cefr_level"],
                "topic": row["topic"],
                "sublevel": row["sublevel"],
                "pos": oxford_pos,
                "phonetic": (chosen.get("pronunciations_raw") or [{}])
            }
            [0].get("text", ""),
            "definition_en": draft.get("definition_en", ""),
            "sentence_en": draft.get("sentence_en", ""),
            "definition_zh": "",
            "translation_zh": "",
            "mnemonic": draft.get("mnemonic", "")

```

```

        "distractor_candidates": dm[:10],
        "wordnet_candidates": wn[:5],
        "source_audit": [
            build_source_audit("FreeDictionaryAPI", ["phonetic",
"definition_raw", "example_raw"], chosen),
            build_source_audit("Datamuse", ["distractor_candidates"],
{"headword": headword}),
            build_source_audit("WordNet", ["semantic_check"],
{"headword": headword, "pos": oxford_pos}),
        ]
    }
    packages.append(package)
return packages

```

The concrete reason to do it this way is that **the raw external source and the final learner-facing text should never be the same field**. That separation makes later legal cleanup, editorial replacement, and quality review much easier. ³⁰

Sample CSV schemas

A compact but practical staging schema:

```
word_id,headword,pos,cefr_level,topic,sublevel,sequence_in_level,source_name,source_url,license_n
```

A practical shipping schema for `words.csv`:

```
word_id,headword,pos,cefr_level,level_number,topic,sublevel,phonetic,definition_en,definition_zh,
```

A practical schema for `word_options.csv`:

```
question_id,word_id,question_type,stem_en,correct_answer,distractor_1,distractor_2,distractor_3,e
```

A practical schema for `audio_manifest.csv`:

```
word_id,voice_provider,voice_name,input_text,output_path,text_kind,sha256,generated_at_utc
```

Sample JSON package

```

{
  "word_id": "lv1001_family_n_0001",
  "headword": "family",

```

```

    "pos": "n",
    "cefr_level": "A1",
    "level_number": 1,
    "topic": "people_and_home",
    "sublevel": "daily_life_core",
    "sequence_in_level": 1,
    "phonetic": "/ˈfæməli/",
    "definition_en": "a group of people like parents and children who belong
together",
    "definition_zh": "家人;家庭",
    "sentence_en": "My family eats dinner together every night.",
    "translation_zh": "我的家人每天晚上一起吃晚饭。",
    "target_span": "family",
    "mnemonic": "Think of a family photo with everyone standing together.",
    "root_affix": null,
    "word_forms": ["families"],
    "distractors": ["class", "office", "team"],
    "pronunciation_path": "audio/en-US/lvl001_family_n_0001.mp3",
    "source_audit": [
      {
        "source_name": "FreeDictionaryAPI",
        "source_url": "wiktionary-page-url",
        "license_name": "CC BY-SA 4.0",
        "retrieved_at_utc": "2026-06-14T00:00:00Z",
        "used_for_fields": ["definition_raw", "translation_candidates",
"phonetic_candidate"],
        "review_status": "approved"
      },
      {
        "source_name": "Datamuse",
        "retrieved_at_utc": "2026-06-14T00:00:02Z",
        "used_for_fields": ["distractor_candidates"],
        "review_status": "approved"
      }
    ]
  }
}

```

Licensing, attribution, and compliance

The practical legal line between prototype and production

The safest way to think about the free/open stack is this:

For an **internal pilot or openly attributed beta**, you can use Wiktionary-derived content and CC-CEDICT if you keep clear provenance, show attribution, and accept that those fields may carry **CC BY-SA obligations**.

FreeDictionaryAPI explicitly requires a visible attribution to FreeDictionaryAPI.com and a link back to the original Wiktionary page, while CC BY-SA requires attribution and ShareAlike for adaptations. ²

For a **closed commercial production app**, do not assume that “AI rewriting” automatically dissolves ShareAlike risk. Your product/legal team should treat Wiktionary- and CC-CEDICT-derived text as potentially copyleft-sensitive. The operationally safer choice is either to keep those fields openly attributed and compatible, or to replace them with **original editorial text** or **licensed proprietary dictionary data** before wide release. This is a product-risk recommendation, not legal advice. ³¹

Attribution templates

Use attribution text that is short, explicit, and reproducible.

For FreeDictionaryAPI / Wiktionary-derived fields

Some lexical data and translation candidates are sourced from Wiktionary via FreeDictionaryAPI.com, licensed under CC BY-SA 4.0. Original source links are available per entry.

This matches FreeDictionaryAPI’s stated data source and attribution expectations. ⁵

For direct Wiktionary dump usage

Some lexical source material is derived from Wiktionary, licensed under CC BY-SA 4.0. Changes may have been made. Source links are recorded in the entry audit log.

Wiktionary pages note that definitions and other text are available under Creative Commons Attribution-ShareAlike. ¹⁴

For CC-CEDICT

Chinese validation data includes CC-CEDICT, licensed under CC BY-SA 4.0.

That is consistent with the CC-CEDICT release page. ¹⁵

For Datamuse

Related-word and distractor candidate generation uses the Datamuse API.

Datamuse asks users to acknowledge the API in publicly available app documentation. ¹⁰

For WordNet

Semantic relations are informed by Princeton WordNet 3.0.

WordNet requires preservation of the notice and proper citation when used in software or interfaces. ⁶

Recommended `source_audit` structure

Store a `source_audit` JSONB array per word package and, optionally, a normalized `source_audit_log` table. A robust record should contain:

```
{
  "source_name": "FreeDictionaryAPI",
  "source_type": "api",
  "source_url": "https://...",
  "retrieved_at_utc": "2026-06-14T03:12:11Z",
  "license_name": "CC BY-SA 4.0",
  "license_url": "https://...",
  "used_for_fields": ["definition_raw", "example_raw", "translations_raw"],
  "content_hash": "sha256:...",
  "review_status": "approved",
  "reviewer": "editor_01",
  "notes": "sense 1 selected; example replaced with original sentence"
}
```

That design is what lets you answer later questions like “Which entries still contain CC BY-SA text?” or “Which examples are original versus source-derived?” without reverse-engineering your database.

Security and compliance checklist

Do not scrape sites when the site itself tells you not to. The MDBG CC-CEDICT page explicitly says **automated or scripted access is prohibited**. For CC-CEDICT, download the official release file once, store it internally, and parse the file locally; do not hit the MDBG web UI from scripts. ¹⁵

Do not use subtitle corpora as your shipped example-sentence source. OPUS makes OpenSubtitles available as a giant corpus, and OpenSubtitles itself operates DMCA removal procedures. That is enough risk signal that subtitle text should be treated as a **frequency source only**, not copied into the final learner content.

¹⁹

Do not deep-link learner-dictionary site HTML unless you have an API/license. Cambridge has an API developer hub and licensing process, Oxford has a formal API, and Merriam-Webster has formal API terms and branding requirements. If you want their content, use their licensed API routes rather than scraping their consumer sites. ³²

Use your own generated audio files for `pronunciation_path`. Third-party dictionary audio URLs can change or carry usage restrictions. Stable app delivery is much easier if every audio asset is generated into your own bucket and indexed by your own `word_id`. Google and Azure both bill by characters and both have free tiers large enough that a Level 1 pilot costs effectively nothing; local TTS is zero marginal cost. ³³

Budget and pilot timeline

Cost estimates

For **TTS**, your pilot is a rounding error. Google's Text-to-Speech pricing page lists free monthly usage tiers for several voice classes, including **4 million characters** for Standard and WaveNet legacy voices, **1 million** for Neural2, and **1 million** for Chirp 3 HD; Azure's free F0 tier includes **0.5 million characters per month** for neural text-to-speech. For a Level 1 pilot of 20 to 80 words, even if you synthesize one word clip and one short sentence per word, you are still far below those free tiers. ³⁴

Local TTS is the cheapest long-term choice if you can accept slightly lower voice naturalness or additional setup. The archived Piper project is MIT-licensed, but the actively maintained `piper1-gpl` project moved to GPL-3.0 and notes that individual voice model cards can carry their own licensing constraints; eSpeak NG is open source and supports 100+ languages and accents. The operational implication is simple: local TTS can get your pilot to zero cost, but you must check **both engine license and voice-model license** before commercial deployment. ³⁵

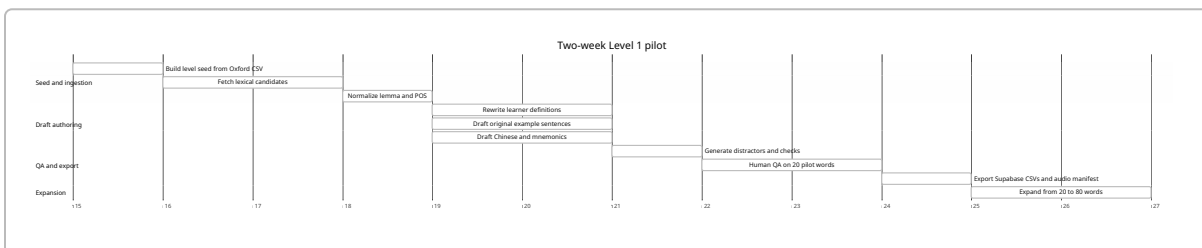
For **paid dictionary APIs**, the only entry-level public production price I found is Oxford. Oxford's sandbox is free but limited to **500 calls** and, in the sandbox, dictionary queries are limited to words beginning with the first letter of the alphabet. Oxford's public paid plan shows **£50/month**, billed annually as **£600**, with **1,000 calls/month** and **£0.05 per extra call**. Merriam-Webster is **free for non-commercial use up to 1,000 queries/day per key**, but commercial use is quote-based. Cambridge issues evaluation keys after approval, and its fees vary by dataset and usage; public production pricing is not posted in the materials reviewed. ³⁶

Item	Public pricing or free tier	What it means for Level 1
Google Cloud TTS	Standard/WaveNet: free up to 4M chars/month ; Neural2 and Chirp tiers have lower but still substantial free tiers, with paid rates shown per 1M chars after that ³⁷	\$0 for a 20–80 word pilot
Azure Speech TTS	F0 free tier includes 0.5M chars/month ; public snippet shows standard neural TTS at \$15 per 1M chars pay-as-you-go, with lower commitment-tier rates for large volume ³⁸	\$0 for a 20–80 word pilot
Piper / eSpeak NG	Software is open source; marginal synthesis cost is local compute only ³⁹	\$0 cash cost, but extra setup and licensing checks

Item	Public pricing or free tier	What it means for Level 1
Oxford Dictionaries API	Sandbox free; paid plan £50/month for 1,000 calls/month ⁴⁰	Cheapest public licensed learner-content upgrade
Merriam-Webster API	\$0 for non-commercial use within 1,000 queries/day ; commercial quote required ⁴¹	Good evaluation option, not a public commercial-price path
Cambridge Dictionary API	Evaluation key after approval; fees vary by dataset/use and are quote-based ⁴²	High-quality learner data, but not budget-transparent

Two-week pilot timeline

The right two-week pilot is not “fill 80 words completely from day one.” It is **prove the pipeline on 20 words**, QA the result, then expand the same workflow to 80 with confidence. That is much safer technically and legally.



A realistic milestone plan is:

1. End of the first week: **20 validated packages** with complete provenance, one question type, and working audio.
2. Early second week: refine validation rules based on real errors.
3. End of the second week: **80 Level 1 packages** exported to Supabase-ready CSVs, plus a clear go/no-go decision on whether you need a paid dictionary license next.

That keeps cost near zero while giving you the evidence needed for a production decision. ²²

Open questions and limitations

The biggest unresolved item is **whether your final commercial app must remain fully proprietary at the content layer**. If the answer is yes, the open stack is still excellent for a pilot, but you should budget later for either licensed learner-dictionary content or a larger original editorial pass. The CC BY-SA obligations around adapted material are real enough that this should be resolved before a wide commercial launch.

The second unresolved item is **dictionaryapi.dev licensing clarity**. The API response shape is attractive, and the project is open source, but the code repository is GPL-3.0 and the site does not clearly state dictionary-content licensing on the landing page materials reviewed. That makes it useful as a bootstrap or fallback source, but not a source I would want to defend as the primary production authority. ¹⁶

The third limitation is that the free/open stack is weaker on **true learner-definition quality** than Cambridge, Oxford, or Merriam-Webster's learner products. The open stack can absolutely get you to a solid pilot, but if your retention or pedagogy depends on highly controlled beginner-friendly dictionary prose, a licensed learner dictionary may be worth the cost earlier than you expect. ⁴⁴

¹ ² ⁴ ⁵ ²² ²⁴ ²⁵ ²⁶ ²⁹ ³⁰ ³¹ **Free Dictionary API**

<https://freedictionaryapi.com/>

³ ⁹ **Frequently Asked Questions | WordNet**

<https://wordnet.princeton.edu/frequently-asked-questions>

⁶ ⁸ **License and Commercial Use of WordNet | WordNet**

<https://wordnet.princeton.edu/license-and-commercial-use>

⁷ **WordNet**

<https://wordnet.princeton.edu/homepage>

¹⁰ ²⁷ **Datamuse API**

<https://www.datamuse.com/api/>

¹¹ ¹³ ¹⁴ **User:Amgine/Wiktionary data & API - Wiktionary, the free dictionary**

https://en.wiktionary.org/wiki/User%3AAmgine/Wiktionary_data_%26_API

¹² **tatuylonen/wiktextextract: Wiktionary dump file parser ...**

https://github.com/tatuylonen/wiktextextract?utm_source=chatgpt.com

¹⁵ **CC-CEDICT download - MDBG Chinese Dictionary**

<https://www.mdbg.net/chinese/dictionary?page=cc-cedict>

¹⁶ ¹⁷ **Free Dictionary API**

<https://dictionaryapi.dev/>

¹⁸ **There was no free Dictionary API on the web when I ...**

https://github.com/meetdeveloper/freedictionaryapi?utm_source=chatgpt.com

¹⁹ ²⁰ **OPUS - Corpora**

<https://opus.nlpl.eu/>

²¹ **opensubtitles.com**

https://rb2.opensubtitles.com/hu/dmca.html?utm_source=chatgpt.com

²³ ⁴³ **Deed - Attribution-ShareAlike 4.0 International**

https://creativecommons.org/licenses/by-sa/4.0/deed.en?utm_source=chatgpt.com

²⁸ **API:Query - MediaWiki**

<https://www.mediawiki.org/wiki/API%3AQuery>

³² **Cambridge Dictionaries Online API Developer Hub**

https://dictionary-api.cambridge.org/?utm_source=chatgpt.com

33 34 37 **Review pricing for Text-to-Speech | Google Cloud**

<https://cloud.google.com/text-to-speech/pricing>

35 39 **rhasspy/piper: A fast, local neural text to speech system**

https://github.com/rhasspy/piper?utm_source=chatgpt.com

36 40 **Oxford Dictionaries API - Sandbox getting started**

https://developer.oxforddictionaries.com/documentation/getting_started/sandbox

38 **Pricing - Azure Speech in Foundry Tools | Microsoft Azure**

<https://azure.microsoft.com/en-us/pricing/details/speech/>

41 **Merriam-Webster Dictionary API**

https://dictionaryapi.com/?utm_source=chatgpt.com

42 **Apply**

https://dictionary-api.cambridge.org/apply?utm_source=chatgpt.com

44 **Merriam-Webster's Learner's Dictionary with Audio**

https://dictionaryapi.com/products/api-learners-dictionary?utm_source=chatgpt.com